

Practical : 8	Implementation of Graph Traversal and Searching using DFS and BFS.
Date : 12/08/2026	

Objective :- To implement Depth-First Search (DFS) and Breadth-First Search (BFS) traversal of a graph in C++ using an adjacency-list representation, and to analyse the time and space complexity of both approaches.

Program :-

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;
// ----- BFS Traversal (Adjacency List, O(V + E)) -----
vector<int> bfsTraversal(int V, vector<vector<int>>& adj, int start) {
    vector<bool> visited(V, false);
    vector<int> order;
    queue<int> q;
    visited[start] = true;
    q.push(start);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : adj[u]) {
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }
    return order;
}
// ----- DFS Traversal (Adjacency List, O(V + E)) -----
void dfsUtil(int u, vector<vector<int>>& adj, vector<bool>& visited, vector<int>& order) {
    visited[u] = true;
    order.push_back(u);
    for (int v : adj[u]) {
        if (!visited[v])
            dfsUtil(v, adj, visited, order);
    }
}
vector<int> dfsTraversal(int V, vector<vector<int>>& adj, int start) {
    vector<bool> visited(V, false);
    vector<int> order;
    dfsUtil(start, adj, visited, order);
    return order;
}
int main() {
    int V = 6;
    vector<vector<int>> adj(V);
```

```
vector<pair<int,int>> edges = {{0,1},{0,2},{1,3},{2,3},{3,4},{4,5}};
for (auto& e : edges) {
adj[e.first].push_back(e.second);
adj[e.second].push_back(e.first);
}
vector<int> bfsOrder = bfsTraversal(V, adj, 0);
vector<int> dfsOrder = dfsTraversal(V, adj, 0);
cout << "BFS Traversal from vertex 0: ";
for (size_t i = 0; i < bfsOrder.size(); i++)
cout << bfsOrder[i] << (i + 1 < bfsOrder.size() ? " -> " : "\n");
cout << "DFS Traversal from vertex 0: ";
for (size_t i = 0; i < dfsOrder.size(); i++)
cout << dfsOrder[i] << (i + 1 < dfsOrder.size() ? " -> " : "\n");
return 0;
}
```

Output :-

BFS Traversal from vertex 0: 0 -> 1 -> 2 -> 3 -> 4 -> 5
DFS Traversal from vertex 0: 0 -> 1 -> 3 -> 2 -> 4 -> 5

Analysis:-

Time & Space Complexity (V = vertices, E = edges):

Method	Time Complexity	Space Complexity
BFS - Adjacency List + Queue	$O(V + E)$	$O(V)$
DFS - Adjacency List + Recursion	$O(V + E)$	$O(V)$

Breadth-First Search and Depth-First Search both systematically visit every reachable vertex of a graph exactly once, but they differ in the order they explore: BFS uses a queue to expand the graph level by level outward from the start vertex, while DFS uses a stack (implemented here via recursion) to plunge as deep as possible along one branch before backtracking. Both algorithms, when implemented with an adjacency list, visit each vertex once and examine each edge at most twice (once from each endpoint), giving an identical $O(V + E)$ time bound; the space cost is also $O(V)$ for both, coming from the visited array plus the explicit queue for BFS or the recursion call stack for DFS. On the sample 6-vertex graph, starting from vertex 0, BFS visits the vertices in the order 0, 1, 2, 3, 4, 5 — exploring both of 0's neighbours before going further — while DFS visits 0, 1, 3, 2, 4, 5, following the $0 \rightarrow 1 \rightarrow 3$ branch all the way down before backtracking to pick up vertex 2. BFS is the natural choice when the shortest path in an unweighted graph is needed, since it always reaches a vertex via the fewest possible edges, while DFS is often simpler for tasks such as cycle detection, topological sorting, or exploring connected components.

Conclusion :- Graph traversal using both BFS and DFS was successfully implemented in C++ on an adjacency-list representation of an undirected graph. The program correctly produced the BFS order 0 -> 1 -> 2 -> 3 -> 4 -> 5 and the DFS order 0 -> 1 -> 3 -> 2 -> 4 -> 5 from the same start vertex on the sample graph, confirming the expected $O(V + E)$ time and $O(V)$ space behaviour for both algorithms. BFS is preferred when the shortest path in terms of edge count is required, while DFS is often more natural for structural tasks like cycle detection and connectivity analysis.